

AppExchange Security Review Submission

This document is prepared exclusively for AppExchange Security Review. It describes the internal technical architecture of the Data Vault managed package including security controls, data flow, and Salesforce platform integration patterns. All architectural claims are directly reflected in the deployed package source code.

. Table of Contents

1 Technical Overview

- 1.1 High-Level Architecture Summary
- 1.2 Design Principles
- 1.3 Multi-Tenant Considerations

2 Technology Stack

- 2.1–2.6 Apex, Scheduler, Named Credentials, S3, JSON, SOQL

3 Core Components

- 3.1 Apex Classes (all 15 production classes)
- 3.2 Custom Settings
- 3.3 Custom Metadata Type
- 3.4 Custom Object – DataVault_Error_Log__c
- 3.5 Lightning Web Component
- 3.6 Permission Set
- 3.7 Custom Labels

4 End-to-End Execution Flow

- 4.1 Step-by-Step (13 steps: config → schedule → batch → S3 → logging)

5 Scheduling Architecture

6 Data Handling & Processing

7 Security Architecture

- 7.1–7.9 Credentials, CRUD/FLS, Sharing, SOQL Injection, Callout Security...

8 Error Handling & Logging

9 Governor Limits & Optimization

10 Scalability & Extensibility

11 AppExchange Review Readiness

1. Technical Overview

1.1 High-Level Architecture Summary

Data Vault is a Salesforce managed package that performs scheduled, incremental backups of Salesforce SObject records to AWS S3. The architecture is intentionally minimal and secure: it uses no external dependencies beyond Salesforce-native Named Credentials, relies entirely on the Salesforce platform for authentication and credential management, and stores no secrets of any kind inside the package.

Core execution path:

```
User (LWC UI)
  → DataVaultExportConfigController (Apex @AuraEnabled Controller)
  → DataVaultExportConfigLogic      (Business Logic Layer)
  → DataVaultExportScheduler        (Schedulable - daily 11:59 PM UTC)
  → DataVaultExportBatch            (Batch Apex + Callout)
  → AWS S3                          (via Named Credential - platform-resolved)
  → DataVault_Error_Log__c          (on any failure)
```

All data operations — SOQL queries, DML inserts, and record updates — are performed with full CRUD and FLS enforcement. Every Apex class is declared **with sharing**, and all SOQL queries include **WITH USER_MODE** to respect org-level sharing rules and field-level security.

1.2 Design Principles

Principle	Implementation
Zero credential storage	Named Credential API name only — no URLs, keys, or tokens stored in package
Least privilege	Permission Set grants only necessary object, field, class, and tab access
Defense in depth	CRUD checks + FLS stripInaccessible() + WITH USER_MODE SOQL + with sharing
Incremental by default	Timestamp watermark (Last_Run_Completed__c) prevents redundant data export
Fail-safe logging	Error buffer never throws; logs always attempted even under DML limits
Single org-level config	One Hierarchy Custom Setting record per org — no tenant cross-contamination
Async execution	Batch Apex isolates callout and DML from synchronous user sessions

1.3 Multi-Tenant Considerations

- **Config isolation:** DataVault_Export_Configs__c is a Hierarchy Custom Setting scoped to SetupOwnerId = OrganizationId. Each subscriber org gets its own configuration record — no shared state between orgs.
- **No static state:** No static variables persist across transaction boundaries or between subscriber orgs.
- **Namespace-safe:** All components are prefixed/namespaced. No hardcoded record IDs anywhere in the codebase.

- **Sharing enforced:** with sharing on all classes ensures each org's record-level sharing model is respected.
- **Governor limits respected:** All batch, callout, DML, and SOQL operations include proactive limit checks to prevent failures cascading across tenants.

2. Technology Stack

2.1 Apex

Apex is the only server-side execution environment available in Salesforce. It provides direct access to SOQL, DML, HTTP callouts, and platform scheduling primitives. Data Vault uses Apex exclusively for all business logic — no JavaScript remoting, no external middleware.

- Enforces CRUD/FLS before every data operation
- Constructs FLS-safe dynamic SOQL using `Schema.describe()`
- Serializes SObject records to JSON for S3 upload
- Manages the batch lifecycle (start → execute → finish)
- Handles all error capture and log persistence

2.2 Salesforce Scheduler (Schedulable Interface)

Salesforce's built-in scheduler is the only way to reliably execute recurring server-side logic inside the platform. It is fully managed, fault-tolerant, and does not require any external trigger or cron service.

- `DataVaultExportScheduler` implements `Schedulable`, scheduled via `System.schedule()`
- CRON: "0 59 23 * * ?" — daily at 11:59 PM UTC
- Single entry point for all automated export runs regardless of cloud platform
- Delegates to `Database.executeBatch(new DataVaultExportBatch(), 200)`

2.3 Named Credentials

Named Credentials are Salesforce's secure, platform-managed mechanism for storing external endpoint URLs and authentication details. They prevent credential exposure in Apex code and eliminate the need for any secret storage within the managed package.

- Subscriber admin creates the Named Credential in their own Salesforce Setup
- Package stores only the Named Credential API name (an opaque string identifier)
- Endpoint constructed as `callout:{namedCredentialName}/{path}` — platform resolves URL and injects auth headers at runtime
- Managed package is architecturally incapable of reading the actual URL, key, or token

2.4 AWS S3 (Cloud Storage Target)

AWS S3 is the initial and currently only supported cloud storage platform. It provides durable, scalable object storage accessible via standard HTTP PUT — making it straightforward to integrate from Apex callouts.

- Receives JSON payloads via HTTP PUT at path `/ObjectApiName/timestamp.json`
- S3 bucket, endpoint, and IAM configuration entirely managed by the subscriber
- Package has no visibility into bucket structure, policies, or access configuration

2.5 JSON Serialization

JSON is the universal interchange format for REST APIs including S3 object uploads. Salesforce's `JSON.serialize()` natively handles SObject serialization.

- `JSON.serialize(scope)` called in `DataVaultExportBatch.execute()` on the List
- `Security.stripInaccessible(READABLE, records)` applied before serialization
- Result: JSON array of field-value maps for all accessible fields on each record

2.6 SOQL with CRUD/FLS Enforcement

Salesforce requires that managed packages respect the running user's field- and object-level permissions. Unguarded SOQL can expose data the user is not permitted to see — a violation of AppExchange Security Review requirements.

- `buildSecureQuery()` dynamically constructs SELECT using only `isAccessible()` fields
- Object names validated via `Schema.getGlobalDescribe()` — never raw user input
- All config queries use `WITH USER_MODE` to enforce sharing rules and FLS at DB level
- `Security.stripInaccessible()` applied as second layer on all read/insert/update ops

3. Core Components

3.1 Apex Classes

Class	Sharing	Layer	Responsibility	Interactions
DataVaultExportConfigController	with sharing	UI Controller	Exposes LWC-facing @AuraEnabled methods; bridges LWC and business logic. Intentionally thin — delegates all logic to service/logic layers. AuraEnabled methods: getCloudPlatformOptions() [cacheable], getConfig() [cacheable], getReadableObjectOptions(), validateNamedCredentialConnection(), saveAndStartJobFromMap(), disableOrEnableExport().	DataVaultExportConfigLogic, DataVaultExportConfigService, DataVaultIntegrationService
DataVaultExportConfigLogic	with sharing	Business Logic	Orchestrates config save, validation, scheduling, and enable/disable. saveAndStartJob() validates inputs, persists config with 'insert/update as user', then calls System.schedule('DataVault Daily Export', '0 59 23 * * ?', ...).	DataVaultExportConfigValidator, DataVaultSecurity_UTILITY, DataVaultObjectAccessHelper
DataVaultExportConfigService	with sharing	Read-Only Data Service	All read-only config and metadata queries. Controllers never issue SOQL directly. Queries DataVault_Cloud_Platform__mdt WITH USER_MODE ORDER BY Display_Order__c.	DataVault_Security_UTILITY, DataVaultObjectAccessHelper
DataVaultExportConfigValidator	with sharing	Validation	All input validation before config save. Verifies: cloud platform active in metadata, Named Credential name non-blank, object readable/queryable. Throws AuraHandledException with specific user-facing messages on failure.	DataVaultObjectAccessHelper, DataVault_Cloud_Platform__mdt
DataVaultExportScheduler	with sharing	Schedulable Entry Point	Implements Schedulable. Loads config, checks Is_Export_Enabled__c, calls Database.executeBatch(new DataVaultExportBatch(), 200). No data operations — purely delegation and guard logic.	DataVault_Security_UTILITY, DataVaultExportBatch, DataVault_Error_Logger
DataVaultExportBatch	with sharing	Batch + Callout (Stateful, AllowsCallouts)	Core export logic. start(): builds FLS-safe QueryLocator with incremental WHERE clause. execute(): strips inaccessible fields, serializes to JSON, calls sendToS3(). finish(): computes run status (Success/Partial Success/Failed), updates config 'as user', flushes error log buffer.	DataVaultObjectAccessHelper, DataVaultIntegrationService, DataVault_Error_Logger, DataVault_Security_UTILITY

Class	Sharing	Layer	Responsibility	Interactions
DataVaultIntegrationService	with sharing	HTTP Callout Layer	All HTTP communication with AWS S3. Constructs PUT request: endpoint = callout:{credential}/{object}/{millis}.json, Content-Type: application/json, timeout 120s. Retry logic: max 2 retries for 429/502/503/504.	DataVault_Error_Logger
DataVaultObjectAccessHelper	with sharing	Schema / FLS Utility	Centralized Schema.describe() logic. Prevents SOQL injection — all field names sourced exclusively from Schema describe. buildSecureQuery() validates object via getGlobalDescribe(), builds SELECT of isAccessible() fields only. Caches describe results.	Used by DataVaultExportBatch, DataVaultExportConfigService, DataVaultExportConfigValidator
DataVault_Security_Utility	with sharing	CRUD / FLS Enforcement Hub	Single auditable class for all security checks. CRUD check methods: isConfigReadable(), isConfigUpdatable(), isConfigReadWrite(), isLoggerCreatable(). FLS strip methods: stripConfigReadable/Updatable/Creatable(), stripErrorLogCreatable(). Config retrieval: SELECT ... WITH USER_MODE WHERE SetupOwnerId = :OrganizationId.	Used by all classes that perform DML or SOQL
DataVault_Error_Logger	with sharing	Error Logging Orchestrator	Public logging API. Methods: logError(context), buildContextFromException(e), logCalloutFailure(input), flush(). Formats callout request/response into debug trace. FLUSH_SIZE_THRESHOLD = 100 records. Never throws.	DataVault_Error_LogBuffer, DataVault_Error_LogRecordBuilder, DataVault_Error_LogContext
DataVault_Error_LogBuffer	with sharing	Buffered DML Accumulator	Batches error log records, auto-flushes when buffer >= 100 records or DML limits approach. Checks canSafelyFlush() before every flush. Uses 'insert as user' after stripErrorLogCreatable(). Catches DmlException — never propagates.	DataVault_Security_Utility, DataVault_Error_Log__c
DataVault_Error_LogRecordBuilder	with sharing	Log Record Factory	Builds DataVault_Error_Log__c records from context. Truncates: Error_Message__c to 32,768 chars, Stack_Trace__c to 32,768 chars, class/method names to 255 chars.	DataVault_Error_LogContext, DataVault_Error_Log__c
DataVault_Error_LogContext	with sharing	Error Data Holder	Plain data object holding: errorType, errorMessage, stackTrace, apexClassName, methodName, timestamp. No logic.	Used by DataVault_Error_LogRecordBuilder
DataVault_Error_StackParser	with sharing	Stack Trace Parser	Extracts class and method from stack trace first line format 'ClassName.methodName: line N'. Populates context for targeted log filtering.	Used by DataVault_Error_LogRecordBuilder

Class	Sharing	Layer	Responsibility	Interactions
DataVault_PackageException	with sharing	Typed Package Exception	Extends Exception. Used for config validation failures, security enforcement violations, and integration errors. Allows distinct catch blocks in callers.	Thrown by DataVaultExportConfigValidator, DataVaultObjectAccessHelper

3.2 Custom Settings — DataVault_Export_Configs__c

Type: Hierarchy Custom Setting (org-level). Chosen over Custom Object because it provides built-in org/profile/user scoping and guarantees one config record per org via SetupOwnerId = OrganizationId.

Field	Type	Purpose
Object_API_Name__c	Text(250)	SObject API name to back up
Named_Credential_Name__c	Text(250)	API name of subscriber's Named Credential
Cloud_Platform__c	Text(80)	Platform identifier (e.g. "AWS_S3")
Is_Export_Enabled__c	Checkbox	Controls whether scheduler triggers batch
Last_Run_Completed__c	DateTime	Incremental backup watermark timestamp (UTC)
Last_Run_Success_Count__c	Number	Records successfully exported in last run
Last_Run_Error_Count__c	Number	Records that failed in last run
Last_Run_Status__c	Text(20)	"Success" / "Partial Success" / "Failed"

3.3 Custom Metadata Type — DataVault_Cloud_Platform__mdt

Visibility: Protected. Chosen over a picklist field because Custom Metadata Types are deployable, version-controlled, and extensible without code changes. New cloud platforms can be added as metadata records — not code releases.

Field	Type	Purpose
Api_Value__c	Text(80)	Unique API identifier (e.g. "AWS_S3")
Is_Active__c	Checkbox	Controls visibility in UI platform dropdown
Display_Order__c	Number(3)	Sort order in combobox (lower = first)

Deployed record: DataVault_Cloud_Platform.AWS_S3 (Api_Value__c = "AWS_S3", Is_Active__c = true, Display_Order__c = 1)

3.4 Custom Object — DataVault_Error_Log__c

Sharing Model: ReadWrite. External Sharing: Private. Name Field: Auto-number (ERR-{0000}). Custom Object chosen over Platform Event because it supports standard list views, tab display, SOQL querying, and DML — all required for the Error Logs tab.

Field	Type	Purpose
Error_Type__c	Picklist	"Callout Failure" or "General Apex Exception"
Error_Message__c	Long Text(131,072)	Error detail — truncated to 32,768 chars
Stack_Trace__c	Long Text(131,072)	Apex stack trace or callout debug — truncated to 32,768 chars
Apex_Class_Name__c	Text(255)	Origin class name
Method_Name__c	Text(255)	Origin method name
Timestamp__c	DateTime	UTC error timestamp

3.5 Lightning Web Component — dataVaultExportConfig

API Version 62.0. Targets: lightning__Tab, lightning__HomePage. LWC chosen over Aura/Visualforce as Salesforce's current-generation UI framework with reactive data binding, wire adapters for cached server calls, and native SLDS integration.

- @wire(getCloudPlatformOptions) and @wire(getConfig) — cached, fired on component init
- getReadableObjectOptions() — called imperatively on edit mode entry only (non-cached)
- saveAndStartJobFromMap() — serializes config to Map for Apex
- disableOrEnableExport() — toggles Is_Export_Enabled__c
- isLocked state machine: view mode (locked) / edit mode (unlocked by Edit button)
- Dual-wire loading guard: configLoaded && cloudPlatformOptionsLoaded before spinner clears

3.6 Permission Set — DataVault_Permission_Set

Permission Sets are the AppExchange-standard access mechanism — no profile modifications. Subscribers assign the set to specific users.

Grant Type	Items	Notes
App & Tab Visibility	Data_Vault app; DataVault_Config tab; DataVault_Error_Log__c tab	Visible only
Custom Setting	DataVault_Export_Configs__c	Read + Edit
Custom Metadata	DataVault_Cloud_Platform__mdt	Read
Custom Object	DataVault_Error_Log__c	Create, Read, Edit, Delete — no ViewAll, no ModifyAll
Field Access	All DataVault_Error_Log__c fields	Read + Edit
Apex Classes	All 15 production classes + test infrastructure	Enabled

Notable omissions (intentional): No ViewAll or ModifyAll on any object. No access to standard objects. The package does not modify standard org data beyond what the running user already has permission to read.

3.7 Custom Labels

Label API Name	Value	Purpose
DataVaultPermissionSetName	DataVault_Permission_Set	Used in test setup to look up permission set by name — no hardcoded string
DataVaultTestUserProfileName	System Administrator	Used in test setup to look up profile by name — org-agnostic

4. End-to-End Execution Flow

4.1 Step-by-Step Walkthrough

Step 1 — User Opens Config Tab

LWC `dataVaultExportConfig` loads. Two `@wire` adapters fire in parallel: `getCloudPlatformOptions()` queries `DataVault_Cloud_Platform__mdt WHERE Is_Active__c = TRUE WITH USER_MODE`; `getConfig()` queries `DataVault_Export_Configs__c WHERE SetupOwnerId = :OrganizationId WITH USER_MODE LIMIT 1`. Component renders in locked (read-only) mode once both wires resolve.

Step 2 — User Clicks Edit

`isLocked = false`; form fields become editable. `getReadableObjectOptions()` called imperatively → `DataVaultObjectAccessHelper.getReadableObjectOptions()` enumerates all SObjects via `Schema.getGlobalDescribe()`, filters to `isAccessible()` && `isQueryable()`, returns labeled list for the Object combobox.

Step 3 — User Selects Platform, Named Credential, Object

Cloud Platform combobox populated from metadata (active platforms only). Named Credential name entered as free text matching a credential created by the subscriber admin in Salesforce Setup. Object combobox filtered to user-accessible SObjects only.

Step 4 — User Clicks 'Save & Schedule Export'

LWC calls `saveAndStartJobFromMap({cloudPlatform, namedCredentialName, objectApiName})`. `DataVaultExportConfigController` receives Map, constructs `SaveConfigInput` DTO, delegates to `DataVaultExportConfigLogic.saveAndStartJob(input)`.

Step 5 — Input Validation

`DataVaultExportConfigValidator.validateSaveInputs()`: Cloud platform non-blank and active in metadata; Named Credential name non-blank; Object API name non-blank and `isObjectReadable() = true`. If any check fails: `AuraHandledException` with specific user-facing message. If valid: `DataVaultIntegrationService.validateNamedCredentialConnection()` issues HEAD request to verify the Named Credential resolves.

Step 6 — Config Persisted

DataVault_Security_Utility.isConfigReadWrite() checked (CRUD). If no existing config: insert as user after stripConfigCreatable(). If existing: update as user after stripConfigUpdatable(). Is_Export_Enabled__c set to true. Config record name: 'Default' (org-level).

Step 7 — Scheduler Created

DataVaultExportConfigLogic.scheduleExportJob() calls: System.schedule('DataVault Daily Export', '0 59 23 * * ?', new DataVaultExportScheduler()). No immediate execution. Scheduler fires at next 11:59 PM UTC.

Step 8 — Scheduler Fires (11:59 PM UTC Daily)

DataVaultExportScheduler.execute() runs. CRUD checks via DataVault_Security_Utility. Config queried via getConfigRecord(). Is_Export_Enabled__c verified (if false, execution stops). Database.executeBatch(new DataVaultExportBatch(), 200) called.

Step 9 — Batch Start Phase

DataVaultExportBatch.start(): SOQL limit check; config read for objectApiName, namedCredentialName, Last_Run_Completed__c; currentRunStarted = System.now() captured. DataVaultObjectAccessHelper.buildSecureQuery() constructs FLS-safe query. Incremental WHERE clause: (LastModifiedDate >= {ts} OR CreatedDate >= {ts}) if Last_Run_Completed__c not null. Returns Database.getQueryLocator(query).

Step 10 — Batch Execute Phase (per 200-record chunk)

DataVaultExportBatch.execute(): callout limit check; heap check (< 500KB remaining = skip). Security.stripInaccessible(READABLE, scope) applied. JSON.serialize(scope) called. DataVaultIntegrationService.sendToS3(credential, object, jsonBody) invoked.

Step 11 — HTTP PUT to AWS S3

DataVaultIntegrationService.sendToS3(): PUT request constructed with endpoint = callout:{credential}/{object}/{millis}.json, Content-Type: application/json, timeout 120s. Retry loop (max 2 retries for 429/502/503/504). Platform resolves Named Credential → injects real URL + auth headers before network. 2xx = success count incremented. Non-2xx = logged, error count incremented.

Step 12 — Batch Finish Phase

DataVaultExportBatch.finish(): DML limit check. Last_Run_Status__c = 'Success' (0 errors) / 'Partial Success' (some errors) / 'Failed' (0 successes). Config updated 'as user' with status, counts, Last_Run_Completed__c = currentRunStarted. DataVault_Error_Logger.flush() called — writes all buffered error logs.

Step 13 — UI Reflects Last Run

Next Config tab visit: `@wire(getConfig)` returns updated config. Status badge, success/error counts, and completion timestamp rendered in read-only Last Run section. No credentials or record data exposed in UI.

5. Scheduling Architecture

Scheduler Creation

`System.schedule('DataVault Daily Export', '0 59 23 * * ?', new DataVaultExportScheduler())` is called inside `DataVaultExportConfigLogic.saveAndStartJob()`. This registers a named scheduled job in the subscriber org's Apex Scheduled Jobs list.

Why 11:59 PM UTC

Fires during low-traffic hours in most enterprise time zones; completes before day rollover keeping backup timestamps day-aligned; avoids conflicts with typical midnight-scheduled jobs.

Job Idempotency

Config is always upserted (insert or update) based on existing `SetupOwnerId` lookup. The incremental timestamp (`Last_Run_Completed__c`) is written only in `finish()` to the value captured at `start()` — ensuring the watermark moves forward monotonically. If the subscriber aborts the job, clicking 'Save & Schedule Export' re-creates it.

Failure Recovery

If the batch fails mid-execution, `finish()` still runs and writes partial counts + 'Partial Success' or 'Failed' status. The scheduler does not retry failed batches — the next daily run picks up from the last successful watermark. `DataVault_Error_Logger.flush()` in `finish()` guarantees error logs are persisted before the batch transaction closes.

6. Data Handling & Processing

6.1 FLS-Safe Record Querying

All SOQL for record export is constructed dynamically by `DataVaultObjectAccessHelper.buildSecureQuery()`:

- Object name validated against `Schema.getGlobalDescribe()` — never interpolated from user input
- Field list built exclusively from `Schema.describe().fields.getMap().values()` where `isAccessible() == true`
- Result: `SELECT {accessible_fields} FROM {objectApiName} [optional incremental WHERE]`
- Query executed via `Database.getQueryLocator()` — enables batch chunking across all matching records

6.2 Incremental Backup Logic

```
First run : Last_Run_Completed__c = null
           → No WHERE clause → ALL records exported

Subsequent : WHERE (LastModifiedDate >= {Last_Run_Completed__c}
                   OR CreatedDate    >= {Last_Run_Completed__c})
```

- `currentRunStarted` captured at `start()` — watermark reflects batch begin time, not finish time
- Both `CreatedDate` and `LastModifiedDate` checked — captures newly created AND recently modified records
- Timestamp stored in UTC via `formatGMT('yyyy-MM-dd'T\HH:mm:ss'Z')` — no timezone ambiguity
- Failed run: `finish()` writes 'Failed' status; watermark not advanced — next run re-exports affected window

6.3 Serialization & Payload Structure

`Security.stripInaccessible(AccessType.READABLE, scope)` is applied before `JSON.serialize(scope)`. This ensures the JSON payload contains only fields the running user is permitted to read — enforced at the serialization layer independently of the SOQL field list.

Property	Value
S3 Object Path	<code>/{{ObjectApiName}}/{{timestamp_millis}}.json</code>
HTTP Method	PUT
Body Format	JSON array of SObject field-value maps
Content-Type Header	<code>application/json</code>
Authentication	Resolved by Named Credential at platform level
Chunk Size	200 records per <code>execute()</code> → 1 PUT per chunk

7. Security Architecture

This section is the primary security reference for AppExchange Security Review. It covers all security controls implemented in Data Vault.

7.1 No Credential Storage in Package

The managed package stores zero secrets, tokens, keys, or endpoint URLs. The subscriber admin creates the Named Credential in their own Salesforce Setup — entirely outside the managed package's scope. The package stores only the Named Credential's API name (a plain string identifier like 'MyS3Bucket'). At callout time, the endpoint is constructed as `callout:{namedCredentialName}/...` — Salesforce resolves this at the platform level, injecting the real URL, authentication headers (AWS IAM signature or equivalent), and SSL certificate trust — none of which are visible to Apex code. Auth headers are never logged.

7.2 Named Credential Architecture

Layer	Responsibility	Managed By
Managed Package	Stores Named Credential API name only	Package (read-only string)
Salesforce Setup	Endpoint URL, auth type, certificate trust	Subscriber Admin
Platform HTTP Layer	Injects URL + auth headers at request time	Salesforce Platform
AWS IAM / Credential	Access key, secret key, IAM role, permissions	Subscriber AWS Account

7.3 CRUD/FLS Enforcement — Three Independent Layers

Data Vault implements three independent layers of data access control. All three must pass independently — a failure at any layer halts the operation.

Layer 1 — CRUD Checks (Schema.sObjectType)

SELECT / INSERT / UPDATE / DELETE guard

`isConfigReadable()` / `isConfigUpdatable()` / `isLoggerCreatable()` called before every DML or SOQL operation. If a check fails, operation is aborted and an exception raised.

Layer 2 — FLS Stripping (Security.stripInaccessible)

Field-level guard on DML and JSON payload

`stripConfigReadable/Updatable/Creatable()` and `stripErrorLogCreatable()` called before all DML and serialization. Removes field values the user cannot access — independent of the field list in SOQL.

Layer 3 — WITH USER_MODE SOQL

Database-level sharing + FLS enforcement

All SOQL queries (config, metadata, validator) include WITH USER_MODE keyword. Enforces org sharing rules and FLS at the database level — independent of class sharing declarations.

7.4 Sharing Model — Universal 'with sharing' Declaration

Every one of the 15 production Apex classes is declared **with sharing**. This ensures record-level visibility respects the running user's sharing rules. In Batch Apex (which runs as the scheduling user), sharing is enforced for the user who configured the export. No class uses **without sharing** — there is zero bypass of sharing rules in this package.

7.5 SOQL Injection Prevention

DataVaultObjectAccessHelper.buildSecureQuery() never interpolates user-supplied strings into SOQL:

- Object name validated against Schema.getGlobalDescribe().keySet() — only valid SObject API names proceed
- Field names in SELECT sourced only from Schema.describe() results — no user input reaches field names
- Incremental timestamp formatted via Datetime.formatGmt() and embedded as a literal — system-generated, not user-supplied
- No dynamic SOQL ever accepts free-form user text

7.6 Data Exposure Prevention

- UI never displays Named Credential details, endpoint URLs, or AWS credentials
- ConfigDTO returned to LWC contains only: cloud platform, credential API name, object API name, enable flag, last run statistics
- Error logs capture only error messages, stack traces, class/method names, timestamps — no record data, no credentials
- JSON payloads to S3 include only fields the running user can read (striplInaccessible applied before serialization)
- No record data is stored inside Salesforce after export — records are streamed to S3 and not persisted in the org

7.7 Callout Security

Property	Implementation
Endpoint resolution	callout: prefix — platform handles SSL/TLS termination and certificate trust
Timeout	120,000 ms — prevents indefinite connection hold
Retry scope	Only transient codes (429, 502, 503, 504) — not auth failures (401, 403)
Auth header visibility	Not visible to Apex — injected by platform, never logged
Certificate trust	Platform-managed via Named Credential setup — package has no influence

7.8 Multi-Tenant Isolation

- Config scoped to `SetupOwnerId = UserInfo.getOrganizationId()` — always returns only the current org's config
- No static variables used for cross-request state
- Custom Metadata records marked Protected — subscribers cannot read or modify them programmatically
- Permission Set grants no access to standard org objects or data beyond what the user already has
- No hardcoded record IDs anywhere in the codebase

7.9 Apex API Surface Control

All `@AuraEnabled` methods are on a single class (`DataVaultExportConfigController`) — a single, auditable public surface. Internal service classes have no `@AuraEnabled` methods and cannot be called directly from LWC or external API consumers. `DataVault_PackageException` is used for security violation signals — a typed exception that propagates intentionally, never silently swallowed.

8. Error Handling & Logging

8.1 Logging Framework Pipeline

```
Exception / Callout Failure
→ DataVault_Error_Logger.logError() / logCalloutFailure()
→ DataVault_Error_LogContext      (data holder)
→ DataVault_Error_LogRecordBuilder (sObject construction + truncation)
→ DataVault_Error_LogBuffer.add()  (in-memory accumulation)
→ DataVault_Error_LogBuffer.maybeFlush() (threshold check)
→ [Threshold met OR explicit flush()]
→ DataVault_Security_Utility.isLoggerCreatable() (CRUD check)
→ Security.stripInaccessible(CREATABLE, logs)   (FLS)
→ insert as user [bulk DML]
→ DataVault_Error_Log__c records persisted
```

8.2 Error Capture Points

Location	Error Type	Trigger
DataVaultIntegrationService	Callout Failure	Non-2xx HTTP status after retries
DataVaultIntegrationService	Callout Failure	CalloutException thrown
DataVaultExportBatch.execute()	Apex Exception	Heap limit exceeded, unexpected exception
DataVaultExportBatch.finish()	Apex Exception	DML limit exceeded, config update failure
DataVaultExportScheduler.execute()	Apex Exception	Config unreadable, batch enqueue failure

8.3 Buffer Flush Strategy

Flush Trigger	Condition	Action
Size threshold	Buffer accumulates ≥ 100 records	Bulk insert as user
DML row limit	$\text{usedRows} + \text{bufferSize} \geq \text{limitRows} - 1$	Immediate flush
DML statement limit	$\text{statements} \geq \text{limitStatements} - 1$	Immediate flush
Explicit flush()	Called in DataVaultExportBatch.finish()	Force-write remaining buffer

8.4 Resilience Properties

- DataVault_Error_LogBuffer.flush() catches DmlException internally — logging never throws and never interrupts main execution
- CRUD check (isLoggerCreatable()) performed before every flush — if user cannot create logs, flush is silently skipped

- String truncation in `DataVault_Error_LogRecordBuilder` prevents `StringException` from field length violations
- `DataVault_Error_StackParser` extracts class + method from stack trace — enables targeted tab filtering
- Callout failure logs include truncated request/response detail (max 16KB each) for precise failure reproduction

9. Governor Limits & Optimization

Limit Type	Check Location	Check Logic	Action on Breach
SOQL queries	DataVaultExportBatch.start()	getLimitQueries() - getQueries() >= 1	Throw DataVault_PackageException, log error
SOQL queries	DataVault_Security_Utility.getConfigRecord()	Query limit check before execution	Throw DataVault_PackageException
HTTP callouts	DataVaultExportBatch.execute()	getLimitCallouts() - getCallouts() >= 1	Log error, skip chunk, increment error count
DML statements	DataVaultExportBatch.finish()	getLimitDmlStatements() - getDmlStatements() >= 1	Log error, skip config update
DML rows	DataVault_Error_LogBuffer.canSafelyFlush()	usedRows + bufferSize >= limitRows - 1	Defer flush
Heap size	DataVaultExportBatch.execute()	getHeapSize() < getLimitHeapSize() - 500,000	Log error, skip chunk

Bulkification Strategy

- DataVaultExportBatch processes records in chunks of 200 — one callout per chunk
- Error log buffer accumulates up to 100 records before flushing — single DML per 100 errors
- DataVaultObjectAccessHelper caches describe results per object — eliminates repeated describe calls across execute() chunks
- No record-by-record DML anywhere in the codebase
- Database.getQueryLocator() used for export query — supports up to 50 million records with platform-managed cursor pagination
- JSON.serialize() on a List is a single heap operation — preflight heap check prevents LimitException

Why Batch Apex (not Queueable or Future)

Factor	Batch Apex	Queueable	Future
Large record sets	Native QueryLocator cursor (50M records)	Manual re-enqueue per chunk	Single transaction limit
Callouts allowed	Yes (Database.AllowsCallouts)	Yes (1 per execute)	Yes (1 per invocation)
Stateful counts	Yes (Database.Stateful)	No	No
Multi-chunk execution	Platform-managed	Developer-managed chain	Not applicable

10. Scalability & Extensibility

10.1 Adding New Cloud Providers — Zero Code Change to Core

The architecture is designed so that adding a new cloud provider requires only a metadata record and a new integration class — no changes to the UI, scheduler, config storage, or error logging.

- Deploy a new `DataVault_Cloud_Platform__mdt` record (e.g. `Api_Value__c = 'Azure_Blob'`, `Is_Active__c = true`)
- `getCloudPlatformOptions()` automatically includes it in the UI dropdown — no LWC changes
- Extend `DataVaultIntegrationService` with a platform-specific `send()` method
- Batch class routes to the correct platform via `Cloud_Platform__c` value on the config record
- User flow, config storage, scheduling, and error logging are entirely unchanged

10.2 Scaling for Large Data Volumes

- Batch size of 200 is configurable in `DataVaultExportScheduler` — can be tuned for heap/callout constraints
- `Database.getQueryLocator()` supports up to 50 million records — platform handles cursor management
- Incremental backup bounds each daily run to daily change rate — not total org data volume
- For very high-volume objects: batch size can be decreased to 50 or 100 to reduce per-execute heap

10.3 Future-Proof Design Decisions

Design Decision	Future Benefit
Custom Metadata for platforms	New cloud providers without schema or code changes to core
Hierarchy Custom Setting for config	Per-profile or per-user config levels possible without schema changes
Layered service architecture	Individual layers replaceable without touching UI or scheduler
Buffered error logging	High-volume error scenarios handled without DML limit risk
Named Credential abstraction	Any auth mechanism (OAuth, API key, certificate) supportable per-credential
Typed <code>DataVault_PackageException</code>	Distinct error handling paths for package vs. platform exceptions

11. AppExchange Review Readiness

11.1 Security Review Compliance Matrix

Security Requirement	Data Vault Implementation	Status
No hardcoded credentials	Named Credential API name only — no URLs, keys, or tokens in package	✓ COMPLIANT
CRUD enforcement	Schema.sObjectType checks before every DML and SOQL operation	✓ COMPLIANT
FLS enforcement	Security.stripInaccessible() + dynamic field describe + WITH USER_MODE	✓ COMPLIANT
Sharing model respected	All 15 production classes declare 'with sharing'; zero 'without sharing' usage	✓ COMPLIANT
No SOQL injection	Object/field names from Schema.describe() only; no user input in SOQL	✓ COMPLIANT
No XSS	No Visualforce; LWC {property} binding auto-escaped by framework	✓ COMPLIANT
No sensitive data exposure	No credentials in UI, logs, or error messages	✓ COMPLIANT
Callout security	All callouts via Named Credentials; platform handles SSL/TLS	✓ COMPLIANT
Async safety	Batch Apex with explicit limit checks; stateful error accumulation	✓ COMPLIANT
Permission model	Permission Set with minimum necessary permissions; no ViewAll/ModifyAll	✓ COMPLIANT
Multi-tenant isolation	Org-scoped config; no shared state; no hardcoded IDs	✓ COMPLIANT
Error handling	All exceptions caught, logged, surfaced without exposing internals	✓ COMPLIANT

11.2 Why This Package Is Secure

Structural Credential Guarantee

The package has an architectural guarantee against credential exposure. Named Credentials are resolved by the Salesforce platform at the HTTP transport layer — after Apex code has constructed the request and before the network connection is made. Apex code in this package is architecturally incapable of reading the actual endpoint URL, access key, secret key, or authentication token. The string stored in the custom setting is an opaque identifier, not a credential.

Layered Data Access Integrity

Three independent security layers must each pass before any data operation proceeds: (1) explicit Schema-based CRUD checks, (2) `Security.stripInaccessible()` on all DML and serialization, and (3) `WITH USER_MODE` in all SOQL. A user with object access but no field access will have that field stripped from both the DML operation and the S3 JSON payload.

No Privilege Escalation Path

All classes declare with sharing. There is no path through the codebase that elevates to a without-sharing context. The batch runs as the user who configured the export — not as a system user. The Permission Set grants only the access necessary to operate the app's components with no access to standard org data beyond what the user already has.

Complete Audit Trail

Every error — callout failure, Apex exception, limit breach — is captured with full context (class, method, timestamp, message, stack trace) and persisted to `DataVault_Error_Log__c`. No exported record data or credentials are stored in logs.

11.3 Salesforce Best Practices Adherence

Best Practice	Implementation
LWC over legacy frameworks	LWC (API version 62.0) exclusively — no Aura, no Visualforce
Named Credentials for callouts	No hardcoded endpoints or credentials anywhere in package
Batch Apex for async processing	Correct primitive for volume + callout requirements
Custom Metadata for configuration	Deployable, version-controlled platform configuration
Hierarchy Custom Setting for org config	Correct primitive for single-record org-level settings
Permission Set for access control	AppExchange standard — no profile modifications required
Typed exception class	<code>DataVault_PackageException</code> for intentional package-level signaling
Null-safe describe operations	All <code>Schema.describe()</code> calls include null checks for org-agnostic safety
Auto-number name on error log	Prevents duplicate name conflicts on high-volume error inserts

11.4 Test Coverage

All 15 production Apex classes have corresponding test classes. The test suite covers:

- Happy-path coverage for all major flows: save, schedule, batch run, error logging
- Negative-path coverage: validation failures, CRUD violations, callout failures
- Mock callout implementations: HTTP 200 success, HTTP 500 server error, HTTP 503 unavailable, `CalloutException`

- Test user setup with explicit Permission Set assignment — tests run in realistic permission context
- System.runAs() used for all permission-context-sensitive tests
- Test coverage meets and exceeds the 75% minimum Apex coverage requirement

This document is prepared exclusively for AppExchange Security Review submission. All architectural claims are directly reflected in the deployed package source code. No architectural claim in this document is speculative or aspirational.
